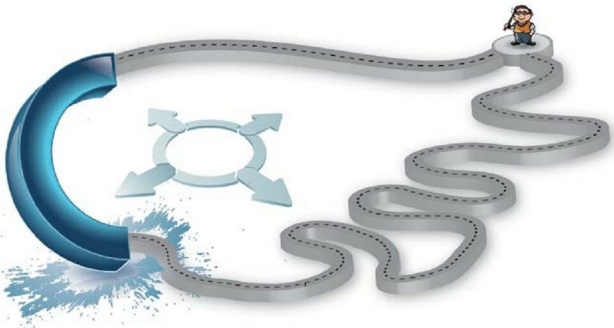




One Extra Step, Courtesy GCC

Compilers fill the gap left by the limited set of constructs in C. We learn some GCC-related tricks in this article.

Being a developer, I'm fascinated by well-structured programs. Although the C programming language has a limited set of constructs, and Dennis Ritchie apparently has attempted to touch everything, sometimes we stumble upon an amazing piece of code, which makes us go: "I wonder if I can write it in C, too!!"

However much smart code you write, with C being blessed with a limited set of constructs, compilers always tend to cross the boundaries there and provide a lot more to the programmers—giving additional functionality, short-hand programming and better performance.

Here we learn some of what is provided by GCC.

Identify the types

The RTTI (Run Time Type Identification) is a feature of the C++ language wherein you can play with the 'types' of variables to create interesting things. This could not be achieved staying within the limits of the C language. However, something of this sort can be attempted at compile time. Consider a simple macro to swap two integers:

```
#define SWAP(a,b) {int temp; temp = a; a=b; b=temp}
```

The same macro could be made available for floats, chars, and even structures and unions by using the following command:

```
#define SWAP(a,b) { typedef (a) temp = (a); a=b; b=temp}
```